CS3400: Data Structures and Algorithms

Intelligent Library Resource Management System

Technical Report

Group Type C: Library Resource Management (books, study rooms, equipment, reservation requests)

Group members	Student number	Responsibility
[Student A name]	[number]	Core data management
[Student B name]	[number]	Request and activity management
[Student C name]	[number]	Advanced data structures and navigation

1. Introduction

1.1 Problem description

A university library manages several kinds of resources: books, study rooms and equipment such as projectors and laptop trolleys. Users request these resources, some requests are more urgent than others, resources change state (available, borrowed, under maintenance), and users must find their way between the library and other campus buildings. Doing this efficiently requires data structures chosen for the job, not just a program that works.

1.2 Objectives

- Store and manage library resources with add, remove, update, display, search and sort operations.
- Process normal requests in first-in-first-out order and urgent requests by priority.
- Support undo of resource changes.
- Retrieve a resource quickly by its ID using a tree and a hash table.
- Model campus locations and pathways as a graph and support traversal, reachability, connected components and shortest paths.
- Save data on exit and load it on start-up.
- Analyse the time and space complexity of the main operations and measure them on large data sets.

1.3 Scope

The system is a menu-driven C++17 console program. Every core data structure is written by the group; the standard library is used only for `string`, input and output streams, and a few maths and time functions. The standard containers (`list`, `queue`, `stack`, `priority_queue`, `unordered_map`, `vector`) are not used. Persistence uses plain text files. A graphical interface, user accounts and a database are outside the scope.

2. System Design

2.1 Architecture

The program is organised in three layers. The container layer holds the generic data structures. The domain layer holds the library resources and requests. The facade class `LibrarySystem` owns one instance of every structure, coordinates them, and implements the menus. Interface and implementation are separated: each class has a header (`.h`) and a source file (`.cpp`). The class templates (`DynamicArray`, `LinkedList`, `Queue`, `Stack`, `PriorityQueue`) and the sorting templates are header-only because C++ requires template definitions to be visible to every file that uses them.

File(s)	Contents
DynamicArray.h	Resizable array template with doubling growth
LinkedList.h, Queue.h, Stack.h	Singly linked list with tail pointer; linked FIFO queue; linked LIFO stack (templates)
PriorityQueue.h	Binary min-heap stored in a DynamicArray (template)
Resource.h/.cpp	Abstract LibraryResource, derived Book, StudyRoom, Equipment, and ResourceFactory
Request.h/.cpp	ResourceRequest with status and priority, and its ordering rule
Sorting.h, Searching.h/.cpp	Insertion, merge and quick sort templates; linear and binary search
Tree.h/.cpp	ResourceBST: self-balancing (AVL) binary search tree keyed by resource ID
HashTable.h/.cpp	ResourceHashTable: separate chaining with the djb2 hash function

File(s)	Contents
Graph.h/.cpp	CampusGraph: weighted undirected graph with adjacency lists
Utils.h/.cpp	Input validation, string helpers, swap template
LibrarySystem.h/.cpp, main.cpp	Facade class with all menus and file handling; program entry point
data/	resources.txt, requests.txt, locations.txt (sample data, also used for saving)

2.2 Class design

LibraryResource is an abstract base class holding the fields every resource shares: ID, name, location, capacity and availability status. Capacity means copies for a book, seats for a study room and units for equipment. Pure virtual functions (`getTypeName`, `getExtraLabel`, `getExtraValue`, `clone`) let each derived class supply its own behaviour: **Book** adds an author, **StudyRoom** adds a floor number and **Equipment** adds a condition note. The program works with `LibraryResource*` everywhere, so printing, saving and searching are polymorphic. **ResourceFactory** builds the right derived object from a type code, which keeps file loading and the add-resource menu free of type-specific code. The virtual `clone()` makes a deep copy through a base pointer, which the undo feature relies on.

ResourceRequest stores the request ID, user name, requested resource ID, submission time, status and priority. Its `operator<` defines urgency: a lower priority number comes first, and for equal priority the smaller (older) request ID comes first, so equal priorities stay first-in-first-out.

LibrarySystem is a facade. It owns the structures and keeps them consistent: every resource is stored once in the linked list, and the tree and the hash table keep extra pointers to the same objects for fast lookup. Only the linked list deletes resources, so there is exactly one owner for each object. The undo stack owns the snapshots it holds until they are consumed or the program ends.

2.3 Memory management and encapsulation

- Data members are private or protected; the public interface exposes only the operations needed.
- Every container allocates with `new` and releases in its destructor. `DynamicArray` follows the rule of three (deep copy, deep assignment, destructor).
- The linked list, queue, stack, tree, hash table and graph have deleted copy operations, so a shallow copy that would cause a double delete cannot happen.
- The undo stack and the catalogue are cleaned up in the `LibrarySystem` destructor.

3. Data Structure Selection

For each structure the table states the problem it solves, why it was chosen, and what else was considered.

Structure	Problem it solves	Why selected	Alternatives considered
Dynamic array	Temporary collections: sorted copies, traversal results, work buffers, heap storage	O(1) indexing and amortised O(1) append; contiguous memory is ideal for sorting and the heap	Fixed array (wastes space or overflows); linked list (no direct indexing)
Singly linked list (head and tail)	The master catalogue of resources, and the processed request history	Resources are added and removed constantly. Insert at the front or end is O(1) with no shifting or reallocation, and the list never has a fixed size	Array (inserting or deleting in the middle shifts elements, growth needs copying); doubly linked list (extra pointer not needed)

Structure	Problem it solves	Why selected	Alternatives considered
Queue (linked)	Normal requests that must be served first-in-first-out	Enqueue and dequeue are both O(1); the structure enforces fairness by construction	Array queue (needs wrap-around logic and a fixed size); sorting by time on every pass (O(n log n))
Stack (linked)	Undo of add, delete and update	The most recent change must be undone first, which is exactly last-in-first-out; push, pop and peek are O(1)	Array stack (fixed size); a list of actions searched backwards
Binary min-heap (priority queue)	Requests that must be served by urgency rather than arrival	Insert and extract-most-urgent are O(log n); the most urgent request is always at index 0	Sorted list (O(n) insert); scanning an unsorted list for the maximum (O(n) per extract)
AVL tree	Ordered access to resources by ID and in-order listing	Library IDs (B001, B002, ...) usually arrive in sorted order, which turns a plain BST into a linked list of height n. AVL rotations keep the height O(log n) in the worst case	Plain BST (O(n) worst case); red-black tree (more complex to implement); sorted array (O(n) insert)
Hash table, separate chaining	Instant lookup of a resource by ID; also used to reject duplicate IDs in O(1)	Average O(1). Chaining never fills up, deleting is simple, and performance degrades gradually when the load factor passes 1. The table size is 53, a prime, which spreads djb2 hash values well	Linear probing (clustering, needs tombstones on delete); double hashing (more complex)
Graph, adjacency lists	Campus locations and pathways	The campus is sparse (each building connects to a few others), so adjacency lists use O(V + E) space	Adjacency matrix ($O(V^2)$ space, wasteful for a sparse graph)

Linked structure versus array. The catalogue is the part of the system where a linked list is the better fit: the number of resources changes all the time, insertion at the beginning, end or any position never moves other elements, and removal only relinks two nodes. Lookup by position is O(n), which is why the system adds the tree and the hash table on top of the list for fast search by ID.

4. Algorithms

4.1 Searching

Linear search checks every element in turn. It works on unsorted data and is used for keyword search on resource names, where the match is a substring and no ordering helps. **Binary search** compares the target with the middle element and discards half of the range each step. It requires sorted data, so the system first sorts a copy of the catalogue by ID with merge sort and then searches it. Binary search pays off when many searches are made on the same sorted collection; for a single search on unsorted data, sorting first costs more than a linear scan. Menu option 3 under Search compares linear, binary, tree and hash lookups for the same ID and reports the number of comparisons and the time of each.

4.2 Sorting

Three comparator-based sorts are implemented as templates. The comparator `isLess(a, b)` returns true when a must come before b, so the same code sorts resources by ID, name, capacity or type, and requests by submission time, priority or status. Every comparator breaks ties with the ID so the result is always well defined.

Algorithm	Idea	Best	Average	Worst	Extra space	Stable
Insertion sort	Insert each element into the sorted prefix by shifting larger elements	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes

Algorithm	Idea	Best	Average	Worst	Extra space	Stable
Merge sort	Split in half, sort each half recursively, merge using a buffer	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick sort	Partition around a middle pivot, recurse on both sides	O(n log n)	O(n log n)	$O(n^2)$	O(log n) stack	No

Merge sort and quick sort both have a much better average case than $O(n^2)$. The quick sort uses the middle element as pivot so that already-sorted input, which is common for IDs, does not trigger the worst case.

4.3 Tree operations

The AVL tree stores resource pointers ordered by ID (left subtree smaller, right subtree larger). After each insert or delete, the nodes on the path back to the root recompute their height and balance factor (height of left minus height of right). If the balance factor leaves the range -1 to +1, a single rotation (left-left or right-right case) or a double rotation (left-right or right-left case) restores balance. Deleting a node with two children replaces it with its in-order successor, the smallest node of the right subtree. Four traversals are provided: in-order (visits IDs in sorted order), pre-order, post-order, and level-order (breadth first, using the group's own queue).

4.4 Priority processing with the heap

A priority queue is the abstract idea of removing the most important item first. A binary heap is one way to implement it: a complete binary tree stored in an array in which every parent is more urgent than its children. Insertion appends at the end and sifts up; extraction takes the root, moves the last element to the root and sifts down. Both take O(log n). Requests are ordered by priority (Emergency, High, Normal, Low) and, within the same priority, by request ID. An emergency request may take a resource that is already borrowed.

4.5 Hashing

The hash function is djb2: start from 5381, then for each character compute hash = hash * 33 + character, and take the result modulo the table size (53). Each bucket holds a linked chain. Insertion first walks the chain to reject a duplicate ID, then adds at the head. The statistics screen reports load factor, empty buckets, collisions and the longest chain.

4.6 Graph algorithms

- **Breadth-first search (BFS)** uses a queue and visits locations level by level. It finds the route with the fewest hops, and it is also used to test reachability.
- **Depth-first search (DFS)** uses recursion (the call stack) and follows one route as deep as possible before backtracking. It is used to find connected components.
- **Dijkstra's algorithm** (array version) repeatedly finalises the closest unvisited location and relaxes its edges, giving the shortest walking distance in metres and the route itself.
- The difference between BFS and DFS: BFS explores nearest locations first and is the right tool when the number of steps matters; DFS explores deeply and is simpler when only visiting or grouping all reachable locations matters.

5. Complexity Analysis

The table analyses the main operations. n is the number of resources, V and E are the numbers of locations and pathways, and the load factor of the hash table is the number of stored items divided by the

number of buckets.

Operation	Structure / algorithm	Time	Case	How the structure affects the result
Enqueue / dequeue a request	Linked queue	O(1)	All cases	Head and tail pointers remove the need to traverse or shift anything
Push / pop an undo record	Linked stack	O(1)	All cases	Only the top node is touched
Insert at beginning / end of catalogue	Linked list with tail	O(1)	All cases	The tail pointer avoids walking to the end
Insert at position k	Linked list	O(k) = O(n)	Worst	Must walk k nodes; no direct indexing
Search resource by name	Linear search	O(n)	Worst (best O(1))	Substring match on unsorted data must examine every item
Search sorted resources by ID	Binary search	O(log n)	Worst (best O(1))	Needs a sorted array; each step halves the range
Lookup by ID	Hash table (chaining)	O(1 + load factor)	Average; worst O(n)	Worst case only if all keys land in one bucket
Insert / search / delete by ID	AVL tree	O(log n)	Worst	Balance guarantees height at most about 1.44 $\log_2 n$; a plain BST would be O(n) worst case
Tree traversal	In/pre/post/level order	O(n)	All cases	Every node is visited once
Insert request / extract most urgent	Binary min-heap	O(log n)	Worst	Height of a complete binary tree is log n
Peek most urgent request	Binary min-heap	O(1)	All cases	The root is at index 0
Sort catalogue	Insertion sort	$O(n^2)$	Average (best O(n))	Shifts elements one place at a time
Sort catalogue	Merge sort	O(n log n)	All cases	Always splits evenly; needs O(n) extra space
Sort catalogue	Quick sort	O(n log n)	Average (worst $O(n^2)$)	Depends on pivot quality
BFS / DFS	Adjacency lists	O(V + E)	All cases	Each vertex and each edge is examined once
Shortest path	Dijkstra, array version	$O(V^2)$	All cases	Selecting the closest vertex scans all V each round; a heap would give O((V + E) log V)

5.1 Measured results

Menu option 10 generates random catalogues of 1,000, 5,000 and 10,000 books, times the three sorting algorithms, and times 1,000 searches with each method. The figures below are from one run, in microseconds of CPU time measured with `clock()`. Exact values vary between machines and runs, but the growth trends are stable.

n	Insertion sort	Merge sort	Quick sort	Linear x1000	Binary x1000	BST x1000	Hash x1000
1,000	1,249	158	123	2,117	183	165	34
5,000	29,605	1,103	823	10,151	232	480	153
10,000	134,591	2,603	1,880	20,989	302	653	250

Going from 1,000 to 10,000 items multiplies insertion sort time by roughly 108, which is consistent with n^2 growth, while merge sort and quick sort grow by roughly 15 to 17 times, consistent with n log n. Linear search grows about tenfold with n, whereas binary search, BST and hash lookups stay almost flat. The

hash lookups show the lowest cost, in line with their average O(1) behaviour.

5.2 Space complexity

- Dynamic array, linked list, queue, stack, heap: O(n).
- Merge sort: O(n) extra for the buffer. Insertion sort: O(1). Quick sort: O(log n) average recursion depth.
- AVL tree: O(n) with one extra integer (the height) per node. Hash table: O(n + buckets).
- Graph with adjacency lists: O(V + E). Dijkstra: O(V) for the distance, predecessor and visited arrays.

6. Testing

The test cases and results are to be added by the group.

7. Challenges and Lessons Learned

7.1 Technical challenges

- **Keeping three structures consistent.** The catalogue, the tree and the hash table all point to the same resource objects. Insertion and removal go through two helper functions that update all three together, and only the linked list ever deletes an object.
- **Undo and object ownership.** A deleted resource is detached rather than destroyed, so the undo stack can put the same object back. For an add or an update, the stack stores a clone made with the virtual `clone()`. Each snapshot has exactly one owner at any time.
- **Balanced tree.** Sequential IDs would have made a plain BST as slow as a list. Implementing the four AVL rotation cases correctly, including in deletion, was the hardest tree task.
- **Fair ordering of equal priorities.** A heap alone does not keep equal-priority items in arrival order, so the comparison falls back to the request ID.
- **Trustworthy benchmarks.** An optimising compiler can delete search loops whose results are never used. The benchmark adds every result to a volatile checksum so the work is really done.
- **Clean shutdown.** If input ends early (for example Ctrl+D), the program catches a dedicated exception, saves its data and exits instead of looping forever.

7.2 Design challenges

- Choosing which parts need linked storage and which need arrays, so every structure is justified by the operations performed on it.
- Separating interface from implementation while keeping the generic containers as header-only templates.

7.3 Team collaboration

[To be completed by the group.]

7.4 Important lessons

- The right structure depends on the dominant operation: a list for frequent insertion, a queue for fairness, a stack for undo, a heap for urgency, a hash table for exact-key lookup, a balanced tree for ordered access.
- Complexity analysis is only meaningful with a stated case (best, average or worst), and measurements confirm it only when the data is large enough.

- Combining structures (list plus tree plus hash table) gives the benefits of each, at the cost of keeping them in step.